

PROGRAM EXPLANATION

プログラム説明資料

荒魂

A R A T A M A

制作期間 2025年10月 - 2026年3月(6ヶ月)

制作形態 チーム制作(11人)

開発環境 C++ / DirectX 11 / HLSL / SDL3

菊池 凌斗

Ryoto Kikuchi / HAL東京 ゲーム学科



作品概要と開発環境

— ゲーム概要

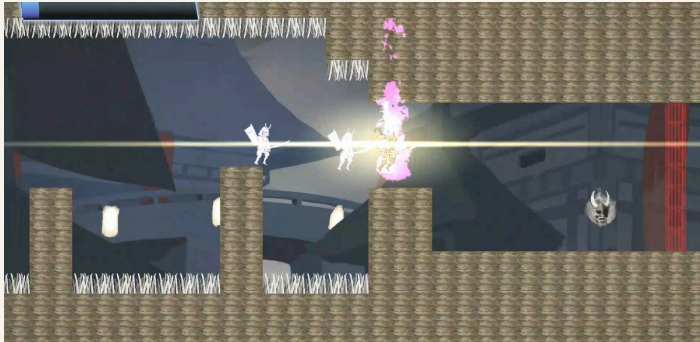
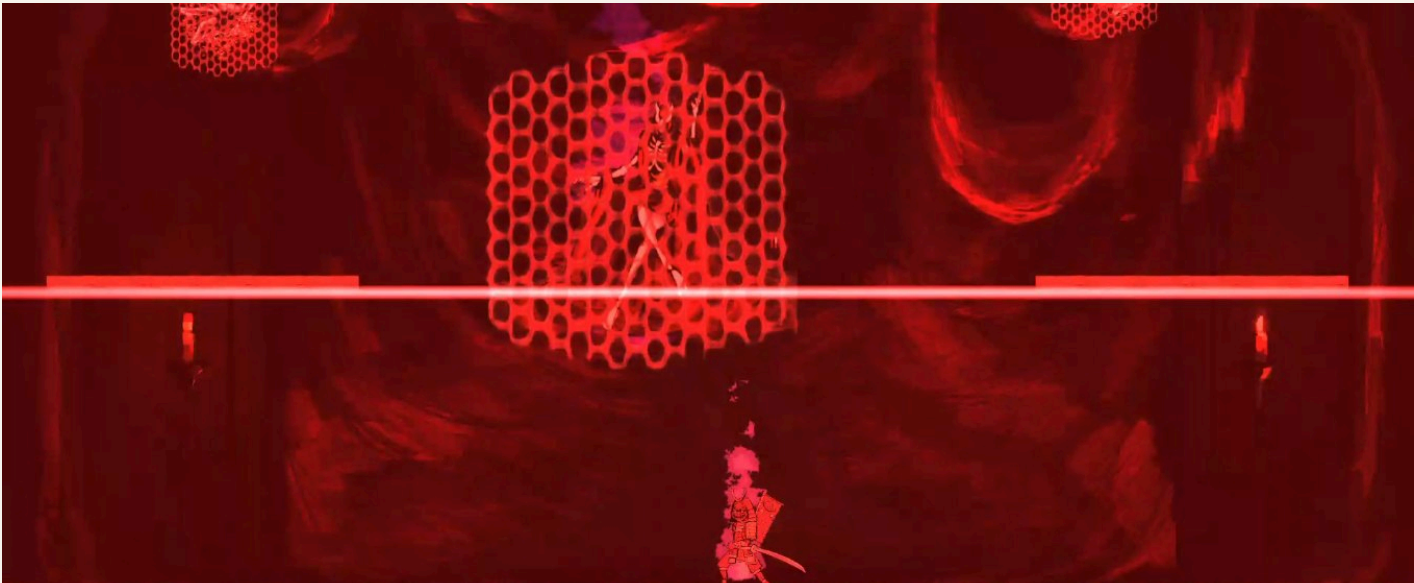
2Dハイスピードの憑依アクション。操作はコントローラーのみ。ステージを進み、ボスと戦う。

— ボス戦の流れ

ボスは通常攻撃を続け、一定時間か一定ダメージで「溜め」に入る。溜め中はバリアと仮面が現れ、すべて壊せばスタン(大ダメージのチャンス)、壊せなければ必殺技が飛んでくる。

— 開発環境

言語 / API	C++ / DirectX 11 (HLSL)
ライブラリ	XAudio2 / Media Foundation / SDL3 / WinInet
期間・形態	2025/10 - 2026/3(6ヶ月)・チーム制作(11人)
私の役割	プログラム(ボスAI・トンネリング防止・オーディオ・敵AI ほか)、チームマネジメント補助



担当範囲 — どのファイルを誰が書いたか

機能	主なファイル	行数	区分
ボスAI	game_boss.cpp/.h game_boss_state.cpp/.h game_boss_attack.cpp/.h game_boss_attacks.cpp/.h	6,810	● 単独
ギミック	game_gimmick.cpp/.h	1,658	● 単独
オーディオ	Audio.cpp/.h audio_def.h	1,318	● 単独
リザルト・ランキング通信	result.cpp/.h result_data.h ranking_loader.cpp/.h	1,107	● 単独
入力・コントローラー	input_manager.cpp/.h game_controller.cpp/.h	935	● 単独
オブジェクト基盤・トンネリング防止	game_object.cpp / game_object.h	785	● 単独 ● 共同
敵AI	game_enemy_ai.cpp/.h	587	● 単独
動画再生	media_player.cpp/.h	461	● 単独
衝突判定の管理(共同)	CollisionManager.cpp/.h	277	● 共同
合計 30ファイル		13,938	

● 単独 ファイルの Author 表記が菊池のみ。

● 共同 他メンバーと共同で作成したファイル(CollisionManager.cpp/.h、game_object.h)。当たり判定のうち、私が担当した箇所はトンネリング防止(game_object.cpp)。

※ 本資料およびGitHubのリポジトリには担当部分のみ掲載。行数はコメント・空行を含む。

音声の追加を「1行」で済ませる — X-Macro

課題

音声をつつすたびに、enum・ファイルパス配列・BGMフラグ配列の3か所を同じ順番で直す必要があった。順番がずれると別の音が鳴るので、追加のたびに確認が要る。

やったこと

音声の定義を audio_def.h の AUDIO_LIST に集約。AUDIO(ID, パス, BGMか) を1行書くだけで、マクロを差し替えて展開し、enum①・パス配列②・BGMフラグ配列を同じリストから生成する(X-Macro)。

ポイント

- 3つの配列が同じ定義から作られるので、インデックスがずれない。
- 現在54件を登録。追加作業は1行。
- 読み込みは拡張子で判別(.mp3 は Media Foundation、それ以外は WAV)。

```
audio_def.h

31 #define AUDIO_LIST \
32     /*=== BGM ===*/ \
33     AUDIO(BGM_TITLE, "resource/sound/BGM/Title.mp3", true) \
34     AUDIO(BGM_GAME1, "resource/sound/BGM/Stage_01.mp3", true) \
```

audio_def.h L31-L34

```
Audio.h

15 #define AUDIO(id, path, isBgm) id,
16 enum class AudioID {
17     AUDIO_LIST
18     MAX
19 };
```

Audio.h L15-L19

```
Audio.cpp

39 #define AUDIO(id, path, isBgm) path,
40 static const char* g_audioFiles[static_cast<int>(AudioID::MAX)] = {
41     AUDIO_LIST
42 };
```

Audio.cpp L39-L42

同じ効果音を連打しても途切れない — SEボイスプール

— 課題

1つの SourceVoice で同じSEを連続再生すると、鳴らすたびに前の音を止めて頭から流し直すため、斬撃音のような連打されるSEが途切れる。

— やったこと

SEごとに SourceVoice を8個①持つプールを作り、呼ぶたびに次の番号の Voice を使うラウンドロビンにした②③。プールはそのSEを初めて鳴らすときに作る(遅延初期化)。作れなかったときは通常再生に切り替わる。

— 音量とフェード

- 最終音量は「マスター × BGM(またはSE) × 個別 × フェード係数」の積。設定画面の音量と演出側の音量を別々に扱える。
- BGMのフェードイン/アウト・クロスフェードは `Audio_Update` が毎フレーム進める自前実装。

```
Audio.cpp

138 static constexpr int SE_VOICE_POOL_SIZE = 8; // 同時再生可能数
139
140 struct SEVoicePool {
141     IXAUDIO2SourceVoice* voices[SE_VOICE_POOL_SIZE] = {}; // Voiceの配列 ①
142     int nextIndex = 0; // 次に使うVoiceのインデックス
143     bool initialized = false; // 初期化済みフラグ
144 };
145
146 // プールから次のVoiceを取得 (ラウンドロビン)
147 IXAUDIO2SourceVoice* voice = g_sePool[idx].voices[g_sePool[idx].nextIndex]; ②
148
149 voice->Stop();
150 voice->FlushSourceBuffers();
151
152 float finalVolume = g_masterVolume * g_seVolume * volume;
153 voice->SetVolume(ClampVolume(finalVolume));
154
155 voice->Start();
156
157 // 次のインデックスへ (ラウンドロビン)
158 g_sePool[idx].nextIndex = (g_sePool[idx].nextIndex + 1) % SE_VOICE_POOL_SIZE; ③
```

Audio.cpp L138 / L140-L144 / L968-L969 / L976-L977 / L989-L995

高速移動でもブロックに食い込まない — 軸分離と脱出探索

— 課題

ダッシュ・落下・ノックバックで1フレームの移動量が大きくなると、ブロックにめり込んだり、挟まれて動けなくなる。

— やったこと

- 移動前後を含む矩形(余白128px)を作り、その範囲のブロック・ギミックだけを取得して判定する①。
- X軸を動かして解決 → Y軸を動かして解決の順に処理②。めり込んだらブロックの縁へ押し戻し、その軸の速度だけ0にする③。もう一方の軸は動き続けるので壁沿いに滑れる。
- それでも詰まったとき(ボス攻撃などに挟まれた場合)は、上下左右と斜めの8方向へ16pxずつ、最大512px探索し、最も近い空き位置へ移す④。

Move(): game_object.cpp L188~ / FindEscapePosition: L70~L186

```
game_object.cpp
79  const float stepSize = 16.0f;
80  const float maxSearchDistance = 512.0f;
...
227 float padding = 128.0f;
228 float minX = std::min(prevPos.x, prevPos.x + desiredMove.x) - padding;
229 float maxX = std::max(prevPos.x + size.x, prevPos.x + size.x + desiredMove.x) + padding;
230 float minY = std::min(prevPos.y, prevPos.y + desiredMove.y) - padding;
231 float maxY = std::max(prevPos.y + size.y, prevPos.y + size.y + desiredMove.y) + padding;
232
233 std::vector<Block*> nearbyBlocks;
234 stage->GetBlocksInRange(minX, minY, maxX, maxY, nearbyBlocks);
...
252 finalPos.x = prevPos.x + desiredMove.x;
...
263         if (desiredMove.x > 0) {
264             finalPos.x = bPos.x - size.x - col0ff.x;
265         } else if (desiredMove.x < 0) {
266             finalPos.x = bPos.x + bSize.x - col0ff.x;
267         }
268         m_velocity.x = 0.0f;
```

game_object.cpp L79-L80 / L227-L234 / L252 / L263-L268

ボスの行動を「状態」ごとのクラスに分ける — State パターン

課題

通常攻撃・溜め・必殺・スタン・撃破を1つの Update に `if` で並べると、状態が増えるたびに壊れやすく読みにくい。

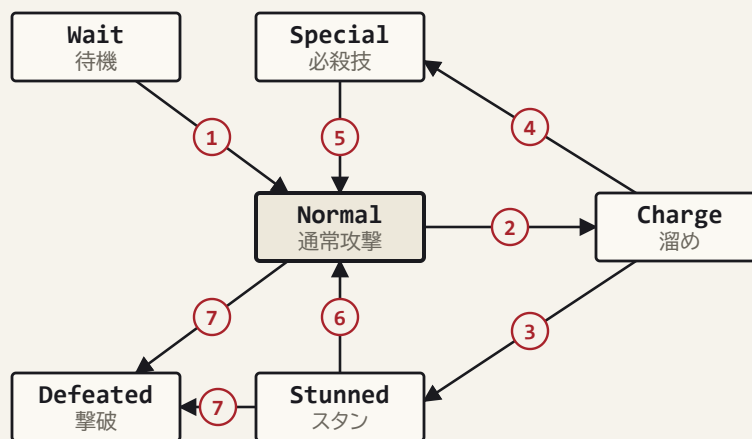
やったこと

状態ごとにクラスを分け、共通の

`IBossState` (Enter / Update / Draw / Exit / OnDamage)を実装。遷移は各状態の Update から `boss->ChangeState(new ...)` を呼ぶだけにした。

ポイント

- ダメージ処理も状態が持つ。スタン中は3倍 (`DAMAGE_MULTIPLIER = 3.0`)。
- 撃破演出は状態の中でさらにフェーズ分割: STAGGER → PLAYER_MOVE → POSSESS_FLASH → SCREAM → DOOR_CLOSE → FADE_OUT。



- 壁の破壊 → プレイヤー入室 → イン트로終了 L52-99
- 12秒経過、または累積ダメージ36 L227
- 仮面をすべて破壊 L279
- 溜め(10秒)が時間切れ L287
- 必殺技が終了 L355
- スタン(5秒)が終了 L423
- HPが0(Normal / Stunned から) L245 / L441

game_boss_state.h

```

26 virtual void Enter(Boss* boss) = 0; // 入った瞬間
27 virtual void Update(Boss* boss, float deltaTime) = 0;
28 virtual void Draw(Boss* boss) = 0;
29 virtual void Exit(Boss* boss) = 0; // 出る瞬間
30 virtual void OnDamage(Boss* boss, int damage) = 0;
31
32 virtual bool CanTakeDamage() const = 0;
33 virtual const char* GetStateName() const = 0;

```

game_boss_state.h L26-L33

攻撃を差し替え可能な部品にする — Strategy パターン

— 課題

攻撃パターンを足すたびにボス本体を書き換えたくない。
同じ攻撃が続くと単調にもなる。

— やったこと

攻撃を `IBossAttack` (Start / Update / Draw / IsFinished / Reset) に抽象化し、通常5種・必殺2種を独立クラスにした^①。`BossAttackManager` は登録された攻撃から1つ選んで実行する。選び方は、乱数を「候補数 - 1」の範囲で引き、**前回の番号以上** **ならずらす方式**^{②③}。再抽選のループなしで、前回と同じ攻撃だけを確実に除外できる。

— 攻撃クラス

通常 GuillotineDrop / MaskBulletHorizontal / MaskBulletDiagonal / MaskBeam / Scratch
必殺 SpecialBeam / SpecialBulletStorm

```
game_boss_attack.h
103 class IBossAttack {
104 public:
105     virtual ~IBossAttack() = default;
106
107     virtual void Start(Boss* boss, Player* target) = 0; // 攻撃開始 ①
108     virtual void Update(float deltaTime) = 0;           // 毎フレーム更新
109     virtual void Draw() = 0;                             // 描画
110     virtual bool IsFinished() const = 0;                 // 攻撃終了したか
111     virtual void Reset() = 0;                           // リセット
```

game_boss_attack.h L103-L111

```
game_boss_attack.cpp
279 // 前回と違うものをランダムに
280 std::uniform_int_distribution<int> dist(0, count - 2);
281 int randIndex = dist(m_rng);
282
283 if (m_lastIndex >= 0 && randIndex >= m_lastIndex) {
284     randIndex++;
285 }
286
287 return randIndex; ② ③
```

game_boss_attack.cpp L279-L287

衝突の最中に壊さない — 遅延破壊

— 課題

仮面は弾が当たった瞬間に壊れる。衝突判定のループの最中に `OnCollision` の中で破壊すると、回っているオブジェクトが消えて参照が壊れ、クラッシュした。

— やったこと

`OnCollision` では `m_pendingDestroy` を立てて Boss に依頼するだけ①。実際の破壊は `Boss::Update` の頭でまとめて実行する②③。

`OnCollision` フラグを立てて Boss に依頼するだけ

▼ 衝突判定のループが終わったあと

`Boss::Update` の頭 `ProcessPendingMaskDestroys()` がまとめて破壊

— ポイント

- 依頼済みの仮面は `m_pendingDestroy` で弾き、二重に壊さない(L1372)。
- 破壊時にプレイヤーへ憑依を通知。全仮面が壊れたらバリア割りのアニメを1回だけ再生する。

```
game_boss.cpp
1372     if (m_isDestroyed || m_pendingDestroy || !otherObject || !otherCollider) return;
1388     if (shouldDestroy && m_owner) {
1389         Audio_PlaySE(AudioID::SE_MASK_BREAK, 0.3f);
1390         m_pendingDestroy = true; // 即座に破壊せずフラグを立てる ①
1391         m_owner->RequestMaskDestroy(this); // Bossに破壊リクエスト
1392     }
434     // マスクの遅延破壊処理 (コリジョン処理完了後に実行)
435     ProcessPendingMaskDestroys(); ②
1184 void Boss::ProcessPendingMaskDestroys() {
1185     if (m_masksToDestroy.empty()) return;
1186     for (auto* mask : m_masksToDestroy) {
1187         if (mask && !mask->IsDestroyed()) {
1189             // 実際の破壊処理
1190             mask->Deactivate(); ③
```

game_boss.cpp L1372 / L1388-L1392 / L434-L435 / L1184-L1190

敵AIは「状態機械1本 + 武器ごとのパラメータ」

— 課題

刀・金棒・弓・槍で別々のAIを書くと同じ処理の複製だらけになる。
壁越しに発見したり、崖から落ちたりするのも避けたい。

— やったこと

- 行動は **PATROL** → **CHASE** → **ATTACK** → **COOLDOWN** の状態機械1本。武器ごとの違いは下表のパラメータの差し替えだけ。
- 視線: 敵→プレイヤーの線上を**32px**ごとに調べ、ブロックかすり抜けブロックに当たれば見えない扱い①②③。
- 崖: 進行方向の足元にブロック・すり抜け床・ギミックが無ければ折り返す(WillFallAhead L308)。

武器	検知	攻撃	追跡速度	攻撃時間	待ち時間
刀	390	280	250	0.4s	2.0s
金棒	350	280	180	0.8s	3.0s
弓	500	400	100	0.8s	2.4s
槍	540	350	220	0.5s	1.8s

距離px・速度px/秒。SetAIType(game_enemy_ai.cpp L33-L112)

```
game_enemy_ai.cpp
461 const float stepSize = 32.0f;
462 int steps = static_cast<int>(distance / stepSize) + 1;
463
464 for (int i = 1; i < steps; ++i) {
465     float t = static_cast<float>(i) / static_cast<float>(steps);
466     float checkX = start.x + dx * t;
467     float checkY = start.y + dy * t;
468
469     // ブロックがあるかチェック
470     if (m_stage->IsBlockAt(checkX, checkY)) {
471         return false; // 障壁がある、見えない
472     }
473
474     // すり抜けブロックもチェック (追加)
475     if (m_stage->IsSlipThroughBlockAt(checkX, checkY)) {
476         return false; // すり抜けブロックも視線を遮る
477     }
478 }
```

game_enemy_ai.cpp L461-L478

外部ライブラリなしでランキングを取得する — WinInet と 302 追跡

— 課題

ランキングはGoogleスプレッドシート(GAS)経由。
GASは `script.google.com` から
`script.googleusercontent.com` へ302で転送する
ため、そのまま GET しても本文が取れない。通信が
失敗・遅延してもゲームは止めたくない。

— やったこと

- WinInet の HTTPS GET で自動リダイレクトを
切り①、3xx のときは `Location` ヘッダを
読んで自前で追跡する(最大10回)②。外部
ライブラリは使っていない。
- リザルト画面では取得を別スレッドで実行し③、
メインスレッドはローディング演出を続ける。
- 15秒で打ち切り、ダミーのランキングを表示して
そのまま進行できるようにした④。
- 送信は `m_submitted` フラグで二重送信を防ぐ。

```
ranking_loader.cpp
72     INTERNET_FLAG_RELOAD |
73     INTERNET_FLAG_NO_CACHE_WRITE |
74     INTERNET_FLAG_SECURE |
75     INTERNET_FLAG_NO_AUTO_REDIRECT, ①
    ...
92     ///--- リダイレクト (301, 302, 303, 307, 308) ---
93     if (statusCode >= 300 && statusCode < 400) {
    ...
98         if (HttpQueryInfoW(hUrl, HTTP_QUERY_LOCATION, locationBuf, &locationSize, NULL)) { ②
    ...
105         currentUrl = newUrl; // 新しいURLで再試行
```

ranking_loader.cpp L72-L75 / L92-L93 / L98 / L105

```
result.cpp
330     g_LoadThread = new std::thread([]() { ③
    ...
336         RankingLoader::GetInstance().LoadFromGoogleSheets();
    ...
371     if (!g_LoadFinished && g>LoadingTimer >= LOADING_TIMEOUT) { ④
372         g_IsTimeout = true;
```

result.cpp L330 / L336 / L371-L372

動画再生 — 受け付けられる形式を探し、自前で RGB に変換する

— 課題

イントロ動画(MP4)を D3D11 のテクスチャとして再生したい。Media Foundation が返せるピクセル形式は環境によって違う。

— やったこと

- 出力形式を RGB32 → YUY2 → NV12 の順に `SetCurrentMediaType` で試し、最初に受け付けた形式を採用する❶❷。
- YUY2 / NV12 はフレームごとに自前で RGB に変換し、整数演算(固定小数点)❸でテクスチャへ書き込む。NV12 は 2×2 ブロックで UV を共有して処理。
- 上下反転して届く動画には、行の読み順を反転して対応。

```
media_player.cpp
69 const GUID formats[] = {
70     MFVideoFormat_RGB32,
71     MFVideoFormat_YUY2,
72     MFVideoFormat_NV12,
73 };
...
81 for (int i = 0; i < 3; ++i) {
82     pType->SetGUID(MF_MT_SUBTYPE, formats[i]);
83     hr = m_pReader->SetCurrentMediaType(
84         MF_SOURCE_READER_FIRST_VIDEO_STREAM, nullptr, pType);
```

media_player.cpp L69-L73 / L81-L84

```
media_player.cpp
317 int D = U - 128;
318 int E = V - 128;
319 int bAdd = (516 * D + 128) >> 8;
320 int gAdd = (-100 * D - 208 * E + 128) >> 8;
321 int rAdd = (409 * E + 128) >> 8;
322
323 int C0 = (298 * (Y0 - 16)) >> 8;
324 int C1 = (298 * (Y1 - 16)) >> 8;
```

media_player.cpp L317-L324

入力デバイスの違いをゲームから隠す

課題

キーボード・マウスとゲームパッド(Xbox / PS / Switch)では入力の形も配置も違う。ゲーム側がデバイスごとに分岐すると破綻する。

やったこと

- 入力をすべて `Command` 構造体(移動・ジャンプ・攻撃・アナログ量・攻撃方向など)に正規化。ゲーム本体は `Command` だけを見る。
- 最後に入力があったデバイスを `InputType` として保持し①、攻撃方向の決め方(マウス位置 / スティック)やUI表示を切り替える。
- スティックはデッドゾーンの外側を 0~1 に再マッピング②して、倒し始めから滑らかに反応させる。
- ボタンは位置(South / East / West / North)で扱い、Xbox・PS・Switch の割り当て差を吸収③。

Command: input_manager.h L19-L47

```
input_manager.cpp
159 if (hasGamepadInput) {
160     inputType_ = InputType::Gamepad; ①
161 } else if (hasKeyboardInput || hasMouseInput) {
162     inputType_ = InputType::MouseKeyboard;
163 }
...
175 if (fabs(cache_.stickX) > STICK_DEADZONE) {
176     float sign = (cache_.stickX > 0.0f) ? 1.0f : -1.0f;
177     float adjusted = (fabs(cache_.stickX) - STICK_DEADZONE) / (1.0f - STICK_DEADZONE); ②
178     command_.moveAnalogX = sign * adjusted;
179 }
```

input_manager.cpp L159-L163 / L175-L179

```
game_controller.h
35 // メインボタン (位置ベース: South/East/West/North)
36 // Xbox: A(下/South) B(右/East) X(左/West) Y(上/North)
37 // PS: Cross(下) Circle(右) Square(左) Triangle(上)
38 // Switch: B(下) A(右) Y(左) X(上)
39 bool buttonDown = false; // South (A / Cross / B) ③
40 bool buttonRight = false; // East (B / Circle / A)
41 bool buttonLeft = false; // West (X / Square / Y)
42 bool buttonUp = false; // North (Y / Triangle / X)
```

game_controller.h L35-L42

調整したい値の置き場を決めておく

調整したいもの	置き場	現在の値・仕組み
武器ごとの敵AI(検知・攻撃距離、速度、待ち時間)	game_enemy_ai.cpp SetAIType() L33-L112	刀 / 金棒 / 弓 / 槍の4セット
ボスが受けるダメージ	game_boss.h DAMAGE_TABLE[武器][通常/特殊] L134	刀・槍・弓は 通常7 / 特殊28、金棒は 通常10 / 特殊40
ボス戦の時間(通常→溜め→スタン)	game_boss_state.h STATE_DURATION / DAMAGE_THRESHOLD / CHARGE_DURATION / STUN_DURATION	12秒 / 累積36 / 10秒 / 5秒。スタン中のダメージは3倍
ハードモード	game_boss.cpp L44, L85 game_boss_state.cpp L310 game_boss_attacks.cpp m_hardModeScale	ボスHP ×1.5 / 攻撃どうしの間隔 ×0.6 / 溜め猶予 ×0.6 / 攻撃内部の間隔・溜め ×0.7
音声の追加	audio_def.h AUDIO_LIST	1行追加(現在54件)

game_boss_attacks.cpp

```
654 m_hardModeScale = GameManager::Instance().IsHardMode() ? 0.7f : 1.0f;
    ...
667 float spawnInterval = SPAWN_INTERVAL * m_hardModeScale;
668 float warningTime = WARNING_TIME * m_hardModeScale;
```

game_boss_attacks.cpp L654 / L667-L668

— やったこと

難易度の切り替えは IsHardMode() のフラグ1つ。攻撃クラスは自分の生成時に m_hardModeScale (通常1.0 / ハード0.7)を決め、間隔・予兆・溜め時間に掛ける。数値を各所へ書き散らさず、調整の入口を決めておく。

担当のまとめと、次の課題

30

掲載ファイル数

— 担当の要点

- ボスAI(State / Strategy / 遅延破壊)、ギミック、オーディオ、敵AI、リザルトとランキング通信、入力、動画再生を担当。
- 当たり判定はチームで分担し、私はトンネリング防止 (`game_object.cpp`)を担当した。
- 設計の型(State / Strategy / X-Macro)を使い、追加・調整がしやすい形を優先した。

13,938

掲載行数(コメント・空行含む)

9

担当した機能

— 次に手を入れたいところ

- **調整値の置き場:** ボスの攻撃間隔やダメージは、コード内の定数とテーブルで持っている。外部ファイル(CSVなど)へ出せば、再ビルドせずに調整できる。
- **トンネリング防止:** 現状は軸分離と脱出探索で実用上の問題は抑えているが、移動経路全体を調べる連続衝突判定(スイープ)ではない。極端な高速移動まで扱うなら検討したい。
- **視線判定:** 32px間隔のサンプリングなので、薄い障害物は見落とし得る。グリッド走査(DDA)で厳密にできる。

3

共同表記のファイル